



Who Is The Killer?

A Pattern Recognition and Machine Learning Investigation

Associate Professor: Dionisios N. Sotiropoulos

1 Background and Story

The Piraeus Vice Homicide Division has released a fully anonymised data set containing **4 000–5 000** crime incidents (homicides and attempted homicides) over the years 2019–2024. Internal intelligence suggests that there exist exactly S **distinct serial killers** (for example $S = 8$) whose activity is hidden in the data.

For a subset of incidents, the responsible killer has been identified with high confidence (e.g. from DNA, video evidence, or confession). For the remaining incidents, the killer is *unknown*.

Your goal in this project is not simply to predict whether a case is solved or unsolved. Instead, you are asked to address the central question:

For each incident in the data set, who is the most likely killer?

You will tackle this problem through a sequence of modelling steps which require, and are designed to consolidate, your understanding of:

- Probability distributions and data exploration,
- Maximum Likelihood Estimation (MLE),
- Bayesian classification,
- Linear classifiers (linear networks),
- Multi-Layer Perceptrons (MLPs),
- Support Vector Machines (SVMs),
- Principal Component Analysis (PCA),
- k -means clustering.

Each section of the assignment reveals a different aspect of the underlying structure. If you follow the questions in order, you will progressively build a strong system for answering “who is the killer?” for each crime.

2 Data Description

You are given a file `crimes.csv` with N rows. Each row corresponds to one incident.

2.1 Mathematical description of the data set

Let

$$X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$$

denote the collection of feature vectors for all N incidents. Each incident i is represented by a d -dimensional column vector

$$\mathbf{x}_i \in \mathbb{R}^d, \quad i = 1, \dots, N.$$

We decompose the feature space into a continuous and a categorical part:

- d_c : the number of continuous features (in our case $d_c = 8$),
- d_{cat} : the effective dimensionality of the categorical block after encoding (e.g. one-hot encoding),
- $d = d_c + d_{\text{cat}}$: the total feature dimension.

For each incident i we write

$$\mathbf{x}_i = \begin{bmatrix} \mathbf{x}_i^{(c)} \\ \mathbf{x}_i^{(\text{cat})} \end{bmatrix},$$

where

$$\mathbf{x}_i^{(c)} \in \mathbb{R}^{d_c} \quad \text{and} \quad \mathbf{x}_i^{(\text{cat})} \in \mathbb{R}^{d_{\text{cat}}}.$$

More explicitly:

- $\mathbf{x}_i^{(c)}$ contains the continuous descriptors (e.g. `hour_float`, `latitude`, `longitude`, `victim_age`, ...).

- $\mathbf{x}_i^{(\text{cat})}$ contains the encoded categorical information (e.g. one-hot encodings of `weapon_code`, `scene_type`, `weather`, `vic_gender`). If the number of possible values for these four categorical variables is C_1, C_2, C_3, C_4 , respectively, then typically

$$d_{\text{cat}} = C_1 + C_2 + C_3 + C_4$$

when full one-hot encoding is used.

2.2 Feature set

As described above, each incident i is represented by a vector $\mathbf{x}_i \in \mathbb{R}^d$ which we decompose as

$$\mathbf{x}_i = \begin{bmatrix} \mathbf{x}_i^{(c)} \\ \mathbf{x}_i^{(\text{cat})} \end{bmatrix},$$

where $\mathbf{x}_i^{(c)} \in \mathbb{R}^{d_c}$ collects the continuous attributes and $\mathbf{x}_i^{(\text{cat})} \in \mathbb{R}^{d_{\text{cat}}}$ collects the encoded categorical attributes. We now list explicitly the components of $\mathbf{x}_i^{(c)}$ and the raw categorical variables that give rise to $\mathbf{x}_i^{(\text{cat})}$.

Continuous descriptors ($d_c = 8$)

The continuous part $\mathbf{x}_i^{(c)}$ consists of the following eight entries, in fixed order:

1. `hour_float`: time of day in hours, real value in $[0, 24)$ (e.g. 23.5 corresponds to 23:30).
2. `latitude`: anonymised latitude coordinate (degrees).
3. `longitude`: anonymised longitude coordinate (degrees).
4. `victim_age`: age of the victim in years.
5. `temp_c`: local air temperature in degrees Celsius.
6. `humidity`: relative humidity in percent.
7. `dist_precinct_km`: Euclidean distance (km) to the nearest police precinct.
8. `pop_density`: population density (persons per km^2) within a fixed radius around the crime scene.

Categorical descriptors and encoding

The vector $\mathbf{x}_i^{(\text{cat})} \in \mathbb{R}^{d_{\text{cat}}}$ contains an encoded representation of the following four raw categorical variables:

- weapon_code** $w_i \in \{0, 1, 2, 3, 4, 5\}$, where for example
0 = knife, 1 = handgun 9mm, 2 = revolver .38, 3 = shotgun, 4 = blunt object,
5 = unknown.
- scene_type** $s_i \in \{0, 1, 2, 3\}$, where
0 = street, 1 = residence, 2 = business, 3 = other.
- weather** $r_i \in \{0, 1, 2, 3, 4\}$, where
0 = clear, 1 = rain, 2 = snow, 3 = fog, 4 = unknown.

`vic_gender` $g_i \in \{0, 1\}$, where 0 = male, 1 = female.

In the raw CSV file, these variables appear as integer codes (w_i, s_i, r_i, g_i) . For most of the models used in this project (linear classifiers, SVMs, MLPs, PCA, k -means), it is convenient to convert these codes to a *one-hot* representation.

If we denote by $C_1 = 6$, $C_2 = 4$, $C_3 = 5$, $C_4 = 2$ the number of possible values for `weapon_code`, `scene_type`, `weather`, `vic_gender`, respectively, then a standard choice is

$$d_{\text{cat}} = C_1 + C_2 + C_3 + C_4 = 6 + 4 + 5 + 2 = 17.$$

In that case, $\mathbf{x}_i^{(\text{cat})}$ is formed by concatenating four one-hot vectors,

$$\mathbf{x}_i^{(\text{cat})} = [\mathbf{e}^{(\text{weapon})}(w_i)^\top, \mathbf{e}^{(\text{scene})}(s_i)^\top, \mathbf{e}^{(\text{weather})}(r_i)^\top, \mathbf{e}^{(\text{gender})}(g_i)^\top]^\top,$$

where, for example, $\mathbf{e}^{(\text{weapon})}(w_i) \in \mathbb{R}^6$ has a single 1 in position w_i and 0 in all other positions.

Throughout the project, you may work either directly with the integer codes (w_i, s_i, r_i, g_i) (for simple counting or plotting), or with the one-hot encoded vector $\mathbf{x}_i^{(\text{cat})}$ when training linear or non-linear models on the full feature vector $\mathbf{x}_i = [\mathbf{x}_i^{(c)}; \mathbf{x}_i^{(\text{cat})}]$.

2.3 Labels and splits

The following additional columns are provided:

- `split` $\in \{\text{TRAIN}, \text{VAL}, \text{TEST}\}$: indicates the partition of the incident.
- `killer_id`: integer in $\{1, \dots, S\}$.

You may also be given a script `score.py` which, given a file with your predictions, will compute accuracy scores on the VAL and TEST splits (using hidden labels).

3 Formal Problem Statement

Assuming that each incident i is represented by a feature vector $\mathbf{x}_i \in \mathbb{R}^d$ as previously discussed, let $K_i \in \{1, \dots, S\}$ denote the true killer identity for incident i . In this context, your final objective is, for each incident i :

- To produce a probability vector

$$\hat{\pi}_i(k) = \mathbb{P}(K_i = k \mid \mathbf{x}_i), \quad k = 1, \dots, S, \quad \sum_{k=1}^S \hat{\pi}_i(k) = 1,$$

representing your posterior belief that incident i was committed by killer k given the observed features $\mathbf{x}_i$.

- To produce a hard classification

$$\hat{c}_i = \arg \max_{k \in \{1, \dots, S\}} \hat{\pi}_i(k),$$

interpreted as your final answer to the question “who is the killer for incident i ?”.

In the final submission file `submission.csv` you must include at least the following columns:

`incident_id`, `predicted_killer`, `p_killer_1`, $\dots$, `p_killer_S`

where `predicted_killer` is your $\hat{c}_i$ and `p_killer_k` denotes the value of the posterior probability $\hat{\pi}_i(k) = \mathbb{P}(K_i = k \mid \mathbf{x}_i)$ for $k = 1, \dots, S$.

During grading, the hidden true labels K_i for the VAL and TEST rows will be used to compute the overall classification accuracy

$$\text{Accuracy} = \frac{1}{N_{\text{eval}}} \sum_{i \in \text{eval set}} \delta_{\hat{c}_i, K_i},$$

where $\delta_{a,b}$ is the Kronecker delta, defined by

$$\delta_{a,b} = \begin{cases} 1, & \text{if } a = b, \\ 0, & \text{if } a \neq b. \end{cases}$$

4 Tasks and Required Methods

This section describes the sequence of tasks you must complete. Each task uses one or more of the core concepts listed in the project title.

Q1. Exploratory distributions (probability distributions)

- Plot histograms for the one-dimensional distributions of `hour_float`, `victim_age`, `latitude`, `longitude` (using TRAIN + VAL).
- For the variable `hour_float`:
 - a) Fit a single Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ using the sample mean and variance.
 - b) Fit a 3-component 1D Gaussian mixture model

$$p(h) = \sum_{j=1}^3 \alpha_j \mathcal{N}(h \mid m_j, s_j^2), \quad \alpha_j \geq 0, \quad \sum_{j=1}^3 \alpha_j = 1,$$

using a suitable algorithm (e.g. EM, or a library implementation).

- c) Plot on the same figure the histogram of `hour_float`, the single Gaussian density, and the mixture density.
- As an additional exploratory step, produce at least one two-dimensional plot involving `hour_float` and one spatial feature (for example `hour_float` vs. `latitude` or `hour_float` vs. `longitude`), using TRAIN+VAL but *without* using any labels. Briefly describe any non-trivial pattern you observe (e.g. curved bands, dense regions, or “gaps”), without yet attempting to explain its cause.

Comment on whether the single Gaussian is adequate, and what the mixture components suggest about different “time-of-day” modes.

Q2. Maximum Likelihood Gaussians per killer (MLE)

Restrict to the TRAIN split, where `killer_id` is known.

For each killer k define

$$\mathcal{I}_k = \{i : \text{split}_i = \text{TRAIN}, \text{killer_id}_i = k\}, \quad N_k = |\mathcal{I}_k|.$$

Assume that, conditionally on the killer k , the continuous features are Gaussian:

$$\mathbf{x}_i^{(c)} \mid (K_i = k) \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

a) Derive the Maximum Likelihood Estimators

$$\hat{\boldsymbol{\mu}}_k = \frac{1}{N_k} \sum_{i \in \mathcal{I}_k} \mathbf{x}_i^{(c)},$$

$$\hat{\boldsymbol{\Sigma}}_k = \frac{1}{N_k} \sum_{i \in \mathcal{I}_k} (\mathbf{x}_i^{(c)} - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}_i^{(c)} - \hat{\boldsymbol{\mu}}_k)^\top.$$

b) Implement the above estimators from scratch (without calling a built-in “fit Gaussian” function). Verify numerically that the log-likelihood produced by your estimates matches (up to numerical tolerance) that of a trusted library.

c) Visualise, for each killer k :

- a heatmap of the covariance matrix $\hat{\boldsymbol{\Sigma}}_k$ for each killer k . Recall that $\hat{\boldsymbol{\Sigma}}_k \in \mathbb{R}^{d_c \times d_c}$ and that its (p, q) entry is the empirical covariance

$$\hat{\Sigma}_k(p, q) = \widehat{\text{Cov}}(X_p, X_q \mid K = k),$$

where X_p and X_q denote two continuous features (e.g. `hour_float`, `latitude`, ...). To construct the heatmap, place the feature indices (or names) along both axes and display the matrix entries $\hat{\Sigma}_k(p, q)$ as colours: large positive covariances appear as intense warm colours, negative covariances as intense cool colours, and values near zero as neutral colours. Optionally, you may also form the corresponding correlation matrix

$$\hat{R}_k(p, q) = \frac{\hat{\Sigma}_k(p, q)}{\sqrt{\hat{\Sigma}_k(p, p)} \sqrt{\hat{\Sigma}_k(q, q)}}$$

and visualise $\hat{R}_k$ as a second heatmap; this highlights which pairs of features are most strongly linearly related for each killer.

- an ellipse for each killer in selected two-dimensional projections, for example `(latitude, longitude)` and `(latitude, hour_float)`, defined as follows: for killer k , in a given 2D projection compute the sample mean vector $\hat{\boldsymbol{\mu}}_k^{(2)}$ and covariance matrix $\hat{\boldsymbol{\Sigma}}_k^{(2)}$ from the TRAIN incidents of that killer in that plane. For each such point $x_i^{(2)}$ compute its Mahalanobis distance

$$D_i^2 = (x_i^{(2)} - \hat{\boldsymbol{\mu}}_k^{(2)})^\top (\hat{\boldsymbol{\Sigma}}_k^{(2)})^{-1} (x_i^{(2)} - \hat{\boldsymbol{\mu}}_k^{(2)}),$$

and set $c_k = \max_i D_i^2$. Then draw the ellipse

$$(x - \hat{\boldsymbol{\mu}}_k^{(2)})^\top (\hat{\boldsymbol{\Sigma}}_k^{(2)})^{-1} (x - \hat{\boldsymbol{\mu}}_k^{(2)}) = c_k,$$

which, by construction, contains all TRAIN points of killer k in that 2D projection.

Q3. Multiclass Gaussian Bayes classifier (Bayesian classification)

Assume a generative model where, for the continuous part,

$$p(\mathbf{x}^{(c)} \mid K = k) = \mathcal{N}(\mathbf{x}^{(c)} \mid \hat{\boldsymbol{\mu}}_k, \hat{\boldsymbol{\Sigma}}_k),$$

with $\hat{\boldsymbol{\mu}}_k, \hat{\boldsymbol{\Sigma}}_k$ taken from Q2, and a prior over killers estimated from TRAIN by

$$\pi_k = \frac{N_k}{\sum_{j=1}^S N_j}, \quad k = 1, \dots, S.$$

For each incident with continuous features $\mathbf{x}_i^{(c)}$, Bayes’ rule gives (unnormalised) posteriors

$$p(K = k \mid \mathbf{x}_i^{(c)}) \propto \pi_k \mathcal{N}(\mathbf{x}_i^{(c)} \mid \hat{\boldsymbol{\mu}}_k, \hat{\boldsymbol{\Sigma}}_k).$$

- a) Implement a multiclass Gaussian Bayes classifier that, given $\mathbf{x}_i^{(c)}$, returns

$$\hat{\pi}_i(k) = \mathbb{P}(K_i = k \mid \mathbf{x}_i^{(c)}), \quad k = 1, \dots, S,$$

and the corresponding hard decision

$$\hat{c}_i = \arg \max_k \hat{\pi}_i(k).$$

- b) Evaluate the classifier on TRAIN (sanity check) and VAL (generalisation):
- report accuracy on TRAIN and on VAL,
 - report the confusion matrix on VAL.
- c) In a two-dimensional projection (for example, the first two principal components from PCA), visualise the decision regions induced by the Bayes classifier, with TRAIN points coloured by true killer.

Q4. Linear classifier (linear networks)

Now treat the problem as a discriminative multiclass classification task. Let $\mathbf{x}_i \in \mathbb{R}^d$ be your full feature vector for incident i (continuous features plus suitable encodings of the categorical features, for example one-hot).

Consider a linear classifier of the form

$$f(\mathbf{x}) = W\mathbf{x} + \mathbf{b}, \quad W \in \mathbb{R}^{S \times d}, \mathbf{b} \in \mathbb{R}^S,$$

with predicted killer for incident i

$$\hat{c}_i = \arg \max_k f_k(\mathbf{x}_i),$$

where $f_k(\mathbf{x}_i)$ denotes the k -th component of the score vector $f(\mathbf{x}_i)$. You may train this linear network with sum-of-squared-errors and one-hot targets.

- a) Train the linear classifier on TRAIN (and choose any regularisation hyperparameters using VAL).
- b) Report VAL accuracy and confusion matrix. Compare with Q3.
- c) In the 2D PCA projection used in Q3, overlay the approximate linear decision boundaries and discuss where the linear model fails to match the Gaussian Bayes classifier.

Q5. Support Vector Machines (SVMs)

Train a non-linear classifier using Support Vector Machines.

- Use a polynomial kernel of degree 2 *or* an RBF kernel.
 - Use a one-vs-rest multiclass strategy.
- a) Train the SVM on TRAIN. Tune hyperparameters (e.g. C and kernel parameters) using VAL.
- b) Report VAL accuracy and confusion matrix. Compare with Q3 and Q4.
- c) In the 2D PCA space, indicate the locations of the support vectors and, where possible, visualise the induced non-linear decision regions.

Q6. Multi-Layer Perceptron (MLP)

Construct a Multi-Layer Perceptron for multiclass classification.

- Input layer dimension equals the feature dimensionality d .
 - At least one or two hidden layers (e.g. 64 and 32 units) with non-linear activations (e.g. ReLU).
 - Output layer has S units with softmax activation.
- a) Train the MLP on TRAIN using cross-entropy loss. Use VAL to choose the architecture (number of hidden units, dropout, etc.) and to perform early stopping.
 - b) Report VAL accuracy and compare with Gaussian Bayes and SVM.
 - c) Compute permutation feature importance on VAL: for each feature, estimate how much the classifier relies on it by measuring the drop in accuracy when its values are randomly permuted:
 - (a) First, compute and store the baseline VAL accuracy A_{base} using the trained MLP and the original validation set.
 - (b) For each feature index $j = 1, \dots, d$ (continuous or categorical), construct a perturbed validation matrix $X_{\text{VAL}}^{(\text{perm}, j)}$ by randomly shuffling *only* the j -th column of X_{VAL} across incidents (i.e. permute its entries), keeping all other columns fixed.
 - (c) Re-evaluate the MLP on $X_{\text{VAL}}^{(\text{perm}, j)}$ and record the new accuracy A_j . Define the importance score for feature j as the drop in accuracy

$$\Delta A_j = A_{\text{base}} - A_j.$$

A large positive ΔA_j indicates that the feature is crucial for correct classification.

- (d) Optionally, repeat the random shuffling several times per feature and average the resulting ΔA_j values to reduce randomness.
- (e) Rank all features by ΔA_j and plot a bar chart of the top 5 most important features, clearly indicating whether they correspond to continuous attributes (e.g. `hour_float`, `latitude`, ...) or to one-hot encoded categorical indicators (e.g. specific weapon, scene or weather categories).

Q7. Principal Component Analysis (PCA)

Now investigate the possibility that killer identities can be separated well in a lower-dimensional “modus operandi” space.

- a) Standardise continuous features (zero mean, unit variance) and one-hot encode categorical features. Run PCA on TRAIN and obtain eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots$.
- b) Plot the eigenvalues λ_j of the PCA covariance matrix as a function of the component index j (with $\lambda_1 \geq \lambda_2 \geq \dots$), and look for an “elbow” in this curve; choose a small integer m such that the first m components explain most of the total variance and justify your choice.
- c) Using the PCA computed on TRAIN, project each VAL feature vector $\mathbf{x}_i$ onto the first two principal components and denote the resulting coordinates by $\mathbf{z}_i = (z_{i1}, z_{i2})^\top$ (PC1 and PC2). Create a scatter plot of the points (z_{i1}, z_{i2}) for all i in the VAL split, colouring each point according to the predicted killer label from your SVM classifier in Q5. Comment on whether you observe approximately S visually distinct groups in this projection (up to some overlap), and how this relates to the separation between killers suggested by your earlier models.

Q8. k -means clustering in latent space

Finally, treat killer identification as an unsupervised clustering problem in the PCA space.

Let m be the number of principal components you selected in Q7.

- a) Using the PCA transformation fitted on the TRAIN split in Q7, project each feature vector $\mathbf{x}_i$ onto the first m principal components. Denote the resulting m -dimensional representation by

$$\mathbf{z}_i = (z_{i1}, \dots, z_{im})^\top \in \mathbb{R}^m.$$

Do this for all incidents in TRAIN, VAL and TEST, but remember that the PCA parameters (mean and principal directions) must be estimated from TRAIN only.

- b) Run k -means with $k = S$ on the set of TRAIN vectors $\{\mathbf{z}_i : i \in \text{TRAIN}\}$. Denote the resulting cluster label for each TRAIN incident i by

$$c_i^{(\text{km})} \in \{1, \dots, S\}.$$

- c) On TRAIN, build a mapping from k -means clusters to killer labels using majority vote. For each cluster index $q \in \{1, \dots, S\}$ define

$$g(q) = \arg \max_{k \in \{1, \dots, S\}} \sum_{i \in \text{TRAIN}} \delta_{c_i^{(\text{km})}, q} \delta_{K_i, k},$$

where K_i is the true killer for incident i in TRAIN and $\delta_{a,b}$ is the Kronecker delta. Intuitively, $g(q)$ is the killer label that appears most frequently among the TRAIN incidents assigned to cluster q .

- d) Evaluate this mapping on VAL as follows. For each VAL incident i :

- assign i to a k -means cluster by applying the trained k -means model to $\mathbf{z}_i$, obtaining $c_i^{(\text{km})}$,
- convert this cluster label to a killer prediction via $\hat{c}_i = g(c_i^{(\text{km})})$.

Compute the VAL accuracy

$$\text{Accuracy}_{\text{VAL}} = \frac{1}{|\text{VAL}|} \sum_{i \in \text{VAL}} \delta_{\hat{c}_i, K_i},$$

where K_i are the (hidden) VAL labels accessible through the scoring script.

- e) Apply the same pipeline to TEST: for each TEST incident i , compute $\mathbf{z}_i$, obtain its cluster label $c_i^{(\text{km})}$ from the trained k -means model, map it to a killer prediction via $\hat{c}_i = g(c_i^{(\text{km})})$, and include these $\hat{c}_i$ as your final unsupervised killer predictions in `submission.csv`.
- f) For visual inspection, project the TEST incidents onto PC1 and PC2 (the first two principal components) using the PCA from Q7, and create a scatter plot of the resulting 2D points coloured by your final predicted killer labels $\hat{c}_i$. Comment on how well the visually observed groups in this plot align with your k -means-based killer assignments.

5 Deliverables and Evaluation

Deliverables

All of the following deliverables are mandatory:

- **Code (per question Q1–Q8):** for each question Q1–Q8 you must provide Python code (either as a Jupyter notebook `.ipynb` or as one or more `.py` scripts) that implements the required computations, generates the requested figures and tables, and can be executed to reproduce your results.
- **Written report (LaTeX):** you must submit a single self-contained report written in LaTeX (both the compiled PDF and the corresponding `.tex` source). The report must be structured by questions Q1–Q8 and include:
 - all mathematical derivations requested in the project,
 - a clear description of your methodology,
 - the main numerical results (tables, metrics),
 - the key figures (plots, visualisations) with captions and discussion.
- **Prediction file (CSV):** you must submit a single CSV file `submission.csv` with at least the following columns:

`incident_id, predicted_killer, p_killer_1, \dots, p_killer_S`

where `predicted_killer` is your hard prediction $\hat{c}_i$ for each incident i , and `p_killer_k` stores the posterior probability $\hat{\pi}_i(k) = \mathbb{P}(K_i = k \mid \mathbf{x}_i)$ for $k = 1, \dots, S$. This file will be used by the grading script to evaluate your performance on the hidden VAL and TEST labels.

- **Short slide deck:** you must also prepare a brief slide deck (e.g. 25 slides) in PDF format summarising:
 - your overall approach and model choices,
 - the most important experimental results,
 - the main visual insights that helped you understand the killer structure in the data.

6 Use of ChatGPT and Other LLMs: Ethics and Disclosure

In this project you are allowed to consult large language models (LLMs) such as ChatGPT, Copilot, Gemini, etc., provided that you follow strict rules of transparency and academic integrity.

Allowed use

You may use LLMs to:

- clarify definitions or concepts (e.g. “Explain PCA in simple terms”),
- obtain high-level guidance (e.g. “How do I implement a Gaussian Bayes classifier in Python?”),
- get suggestions for code structure or debugging hints,
- get feedback on the presentation of your LaTeX report (phrasing, structure).

However, **you remain fully responsible** for:

- checking the correctness of any mathematics, code, or claims suggested by an LLM,
- understanding every step that appears in your final submission,
- being able to reproduce and explain your results independently.

Prohibited use

It is strictly forbidden to:

- submit work (text, derivations, code) that you do not understand, even if it was produced by an LLM,
- hide or misrepresent the extent to which you used an LLM,
- claim as entirely your own an analysis that was largely copied from an LLM without proper checking or adaptation.

Violations may be treated as academic misconduct.

Mandatory disclosure of prompts and responses

If you use any LLM at any stage of the project, you **must** include a complete log of that interaction as part of your submission.

- Create a separate appendix in your LaTeX report (for example, titled “Appendix A: LLM Prompts and Responses”) or a separate PDF file clearly labelled for this purpose.
- For every interaction, include:
 1. the full prompt you entered (exact text),
 2. the full response you received (unedited),
 3. optionally, a brief 1–2 line comment on how you used or modified that response in your work.
- If you had many short interactions, you may group related prompts and responses together, but *none* should be omitted.

How this affects evaluation

- Your grade will be based primarily on the quality, correctness, and depth of *your* analysis (derivations, code, and interpretation).
- The LLM logs are used to verify transparency and to understand how you used such tools.
- Failure to include the LLM prompt/response log, despite having used an LLM, will be considered a violation of the project rules and may result in penalties.

In summary: you may use LLMs as assistants, but not as replacements. You must *fully disclose* how you used them by submitting all prompts and responses together with your other deliverables.

6.1 Evaluation

Your work will be evaluated based on:

- Correctness and clarity of the theoretical derivations (MLE, Bayes classifier, PCA formulation, etc.).
- Quality and correctness of the implemented models and visual analysis.
- Classification accuracy on VAL and TEST for predicting `killer_id`.
- Quality of interpretation: how well you connect the statistical tools to the forensic question “who is the killer?”.

7 Group work policy

You are allowed to work on this project in small groups, subject to the following rules:

- **Maximum group size:** each group may consist of at most **3 students**. You may also work alone or in pairs if you prefer.
- **Single submission per group:** each group submits exactly one set of deliverables (code, LaTeX report, `submission.csv`, slides). code must clearly list:
 - the full name and student ID of each group member,
 - the group size and a contact email for the group.
- **Internal collaboration:** within a group you are expected to collaborate closely (discuss ideas, share code, divide subtasks). However, every group member must understand the complete solution and be able to explain any part of the report or code if asked.
- **No inter-group sharing of solutions:** different groups may discuss high-level ideas, but direct sharing of code, reports, or detailed solutions across groups is not allowed and will be treated as academic misconduct.

Good Luck